The Constrained Dual-Arm Task-Space BiRRT

A technical report on the current CDFM (M1) planner implementation

husky_assembly_tamp `motion_planner` — written for Yijiang Huang

29 September 2026

Abstract

This report explains the bidirectional task-space RRT that plans the M1 "constrained dual-arm free movement" (CDFM): both arms of the Husky dual-UR5e hold one bar rigidly and carry it from a home (loading) pose to the approach pose in front of the structure. The search runs on the *bar pose* in SE(3), and every tree node is lifted to a 12-joint configuration by analytical inverse kinematics as the tree grows. The pseudocode below is written directly from the code in `dual_arm_task_space_rrt/core.py` and `api.py`; lines that differ from the textbook RRT-Connect of Kuffner and LaValle are printed in red and tagged [D-n], with the list of differences collected in Section 5.

Contents

1 What is being planned **2**

2 Where the code lives **3**

3 The textbook baseline: RRT-Connect **3**

4 The implemented search **4**
4.1 Building blocks . . . 4
4.2 Lifting a bar pose to joints . . . 5
4.3 Inside `ssik_arm_branches`: how the IK branches are produced . . . 5
4.4 Growing one tree toward a pose . . . 9
4.5 The bidirectional loop . . . 9
4.6 Stitching the seam . . . 10
4.7 Sampling . . . 11

5 Core differences from the standard BiRRT **11**

6 Around the search: endpoints, stages, smoothing **12**
6.1 Stages . . . 12
6.2 Getting the two endpoints (derive_start=True) . . . 13
6.3 Smoothing . . . 14

7 Parameters and their defaults **15**

8 What the benchmark measured **16**

9 Reading guide **17**

1 What is being planned

The movement. An M1 movement starts with the bar already gripped by both tools and ends with the bar at its approach pose next to the partially built structure. Nothing is grasped or released along the way, so the two tool frames keep a fixed relation to the bar for the whole motion. The robot base does not move.

Variables.

- $q = (q_L, q_R) \in \mathbb{R}^{12}$: the six left-arm joints followed by the six right-arm joints (canonical UR order).
- $T \in \mathrm{SE}(3)$: the world pose of the bar. In code a pose is a PyBullet pair `(pos, quat_xyzw)`.
- $G_L, G_R \in \mathrm{SE}(3)$: the two *grasp transforms*, bar-from-tool0. They are read once from the goal (or start) configuration and never change. The world tool poses implied by a bar pose are $T\,G_L$ and $T\,G_R$.
- $\mathrm{FK}_L(q_L), \mathrm{FK}_R(q_R)$: forward kinematics of each arm to its `tool0` link.

The constraint. A configuration is admissible only when both tools sit exactly on the bar:

$$\mathcal{M} \;=\; \{\, q \in \mathbb{R}^{12} \;:\; \exists\, T \in \mathrm{SE}(3) \text{ with } \mathrm{FK}_L(q_L) = TG_L \text{ and } \mathrm{FK}_R(q_R) = TG_R \,\}. \tag{1}$$

$\mathcal{M}$ is a six-dimensional surface inside the twelve-dimensional joint space; a random joint sample lands on it with probability zero. That is why the planner does not sample joints at all. It samples the bar pose T (six numbers), and *lifts* each T to a $q \in \mathcal{M}$ by solving inverse kinematics for both arms. The only freedom hidden inside that lift is the choice of IK branch (a UR arm has up to eight for one tool pose), and much of the machinery described below exists to keep that choice consistent along a path.

What "solved" means. The output is a list of bar poses $T_0, \ldots, T_n$ with $T_0 = T_{\text{start}}$, $T_n = T_{\text{goal}}$, and the matching joint path $q_0, \ldots, q_n$ such that

(i) every q_i holds the bar at T_i (Eq. 1, verified by FK to 10^{-4}),
(ii) consecutive poses are at most one step (δ_p metres, δ_r radians) apart,
(iii) consecutive configurations are at most τ apart per joint, $\|q_{i+1} - q_i\|_\infty \le \tau$ with $\tau = 10^\circ$ (the *continuity gate*), and
(iv) every q_i is collision-free for the whole cell (robot, tools, attached bar, structure, obstacles).

2 Where the code lives

Function	File	Role
`plan_constrained_dual_arm`	`api.py`	public entry point; endpoints, stage switches, calls the search, then the smoother
`_derive_constrained_start_for_plan`	`api.py`	goal IK, goal/home branch pairing, backward-tracked start, direct-path shortcut
`_ssik_pair_goal_branch_with_home`	`api.py`	pick the goal IK branch pair that a home pose can reach
`plan_pose_birrt`	`core.py`	**the bidirectional search** (Alg. 8)
`extend_toward`	`core.py`	greedy walk of one tree toward a pose (Alg. 7)
`solve_dual_arm_pose_ik`	`core.py`	lift one bar pose to q (Alg. 2)
`joint_step_exceeds_threshold`	`core.py`	the continuity gate
`pose_to_feature_vec`, `nearest_node`	`core.py`	the metric and the nearest-neighbour scan
`sample_pose`	`core.py`	the random bar-pose sampler
`build_cfab_pose_collision_fn`	`core.py`	bar-only collision pre-check (CC.4)
`derive_constrained_start_tracked`	`core.py`	start derivation by tracking goal→home (Alg. 11)
`smooth_dual_arm_pose_path`	`smooth.py`	shortcut smoother in pose space (Alg. 12)
`ssik_arm_branches`, `rebranch_toward_seed`	`ssik_ik.py`	analytical IK branches, 2π re-branching (Alg. 3)
`ssik_inprocess.solve`	`keyframe/`	loads the generated per-arm solver module and forwards the call
artifact `solve`, `_solve_algebraic`	`asset/ssik/`	the generated per-arm ssik solver: candidates, FK gate, dedup, limits (Algs. 4, 5)

All paths are relative to `husky_assembly_tamp/motion_planner/` unless they start with `keyframe/` or `asset/`, which are siblings of `motion_planner/` inside the package; the RRT files sit in the `dual_arm_task_space_rrt/` sub-package. Line numbers quoted in Section 9 refer to the tree at the time of writing. An earlier, Markdown-format review of the same code is in `docs/plan_constrained_dual_arm.md`; the benchmark that measures it is described in `benchmark/README.md` and `benchmark/analysis_night1.md`.

3 The textbook baseline: RRT-Connect

For reference, Algorithm 1 is RRT-Connect as introduced by Kuffner and LaValle (ICRA 2000), which is what people usually mean by "BiRRT". It plans in the configuration space $\mathcal{C}$ with a single validity test (CollisionFree) and a fixed step ε.

Algorithm 1: Standard RRT-Connect (Kuffner & LaValle 2000)

function RRT-Connect($q_{\text{init}}, q_{\text{goal}}$)
  $\mathcal{T}_a \leftarrow \{q_{\text{init}}\}$; $\mathcal{T}_b \leftarrow \{q_{\text{goal}}\}$
  for $k = 1 \dots K$ **do**
    $q_{\text{rand}} \leftarrow$ RandomConfig() ▷ uniform in $\mathcal{C}$
    if Extend($\mathcal{T}_a$, q_{rand}) $\neq$ Trapped **then**
      if Connect($\mathcal{T}_b$, q_{new}) = Reached **then**
        return Path($\mathcal{T}_a$, $\mathcal{T}_b$) ▷ concatenate the two branches at q_{new}
      end if
    end if
    Swap($\mathcal{T}_a$, $\mathcal{T}_b$) ▷ strict alternation
  end for
  return Failure
end function

function Extend($\mathcal{T}$, q)
  $q_{\text{near}} \leftarrow$ Nearest($\mathcal{T}$, q) ▷ Euclidean in $\mathcal{C}$

$q_{\text{new}} \leftarrow q_{\text{near}} + \varepsilon \dfrac{q - q_{\text{near}}}{\|q - q_{\text{near}}\|}$ ▷ one fixed step
    if CollisionFree($q_{\text{near}}, q_{\text{new}}$) **then**
        add vertex q_{new} and edge $(q_{\text{near}}, q_{\text{new}})$ to $\mathcal{T}$
        return Reached if $q_{\text{new}} = q$ else Advanced
    end if
    return Trapped
end function

function Connect($\mathcal{T}$, q)
    repeat
        $S \leftarrow$ Extend($\mathcal{T}$, q)
    until $S \neq$ Advanced ▷ keep stepping until reached or blocked
    return S
end function

Two properties of the baseline matter for the comparison:

- A node *is* a configuration; there is nothing to compute when a node is added, and concatenating the two trees at the meeting node is trivially a valid path.
- Validity is one boolean (collision). There is no notion of a step being *kinematically* inconsistent with the previous one.

4 The implemented search

4.1 Building blocks

Pose interpolation. Interp(T_a, T_b; δ_p, δ_r) (PyBullet-planning `interpolate_poses`) returns $N = \lceil \max(\|p_b - p_a\| / \delta_p,\ \theta(R_a, R_b)/\delta_r) \rceil$ intermediate poses with linear position and slerp orientation, followed by T_b itself. So the last element of the walk is *exactly* the target pose; this is what makes "the trees meet" an equality rather than a tolerance test in the bidirectional planner.

Feature-point metric. The bar's eight bounding-box corners $c_1, \ldots, c_8$ (bar-local; the box is $0.03 \times 0.03 \times 1.0$ m by default) are pushed through a pose to give a 24-vector

$$\phi(T) = [\, Tc_1;\ Tc_2;\ \ldots;\ Tc_8 \,] \in \mathbb{R}^{24}, \qquad d(T_1, T_2) = \|\phi(T_1) - \phi(T_2)\|_2 \,. \tag{2}$$

This single number blends translation and rotation in a way that is scaled by the physical size of the bar (rotating a long bar moves its corners a lot), which avoids having to pick a weight between metres and radians. ϕ is cached per node so nearest-neighbour search is a linear scan over cached 24-vectors.

Continuity gate.

$$\text{Jump}(q', q) \ \equiv\ \|q' - q\|_\infty > \tau, \qquad \tau = \texttt{DEFAULT_JOINT_CONTINUITY_THRESHOLD_RAD} = 10^\circ. \tag{3}$$

The comparison is on *raw* joint values (no wrapping to $\pm\pi$; the unwrapping helper is left in the source but disabled). A step that fails this gate is interpreted as the IK having flipped to a different solution branch, and is discarded.

Two collision predicates.

- `collision_fn`(T): bar-only. The attached bar is moved to the sampled world pose by rewriting its attachment frame, and only compas_fab's step CC.4 (attached rigid body vs. environment) runs. Cheap pre-reject, used in every stage.

- `joint_collision_fn`(q): full cell. Clones the start state, writes the twelve joints, runs the complete compas_fab check (self, tools, attached bar, environment, ACM). Stage 3 only. Measured cost ≈ 13 ms per call (Section 8).

4.2 Lifting a bar pose to joints

Algorithm 2: SolveDualArmPoseIK — `solve_dual_arm_pose_ik` (ssik backend)

function SolveDualArmPoseIK(T, q_{seed})
  for arm $\in \{L, R\}$ **do**
    $X \leftarrow T\,G_{\text{arm}}$ ▷ world tool0 target
    $B \leftarrow$ `ssik_arm_branches`(arm, X, $q_{\text{seed,arm}}$, `allow_rescue=False`)
      every analytical branch inside joint limits, each shifted by $k \cdot 2\pi$ per joint toward the seed when that stays in-limit, sorted by $\|\cdot - q_{\text{seed,arm}}\|_\infty$ (Alg. 3, Section 4.3)
    if $B = \varnothing$ **then return** None
    end if ▷ tool pose unreachable for this arm
    $q_{\text{arm}} \leftarrow B[0]$ ▷ nearest branch to the seed
  end for
  $q \leftarrow (q_L, q_R)$
  if not `validate_dual_arm_bar_pose`(q, T) **then return** None
  end if
    PyBullet FK re-check: both tools within 10^{-4} of TG_L, TG_R, and the two implied bar poses agree
  return q
end function

Two remarks. First, the two arms are independent 6R chains, so the lift is two separate analytical solves; nothing about the second arm depends on the first. Second, "nearest branch to the seed" is the *only* mechanism that keeps a walk on one branch sheet. There is no memory of which branch was chosen earlier beyond the previous configuration; the continuity gate (Eq. 3) is the safety net that catches the cases where nearest still means far. What "branch" means, and how the list B is produced, is the subject of the next subsection.

4.3 Inside `ssik_arm_branches`: how the IK branches are produced

Every IK call in the planner goes through one function, `ssik_ik.ssik_arm_branches`. It is a thin adapter over the analytical solver library *ssik* (v3.2.0 in the planner's venv), and the stack it sits on has four layers:

layer	where	what it does
adapter	`motion_planner/ssik_ik.py`	world → arm-base frame, 2π re-branching toward the seed, raw-distance sort (Alg. 3)
loader	`keyframe/ssik_inprocess.py`	imports the generated per-arm module once and forwards the call
artifact	`asset/ssik/left_ur_arm_ik.py`, `asset/ssik/right_ur_arm_ik.py`	the generated solver: algebraic candidates, FK check, dedup, limits, seed sort (Alg. 4)
library	`ssik/solvers/ikgeo/`, `ssik/postprocess.py`, `ssik/refinement/` (installed package)	Raghavan–Roth elimination, limit filters, perturbation rescue (Algs. 5, 6)

Why it is not the textbook UR solver. The artifact modules were generated once by `ssik build` from the *calibrated* URDF, with each arm rooted at its own `*_ur_arm_base_link` and ending at `*_ur_arm_tool0`. The familiar closed-form UR inverse kinematics (eight postures: shoulder

left/right, elbow up/down, wrist flipped/not) relies on the nominal geometry – axes 2, 3, 4 exactly parallel, axes 4, 5, 6 pairwise perpendicular and meeting. Calibration breaks that. In the left-arm artifact the second joint axis is $(2.7 \times 10^{-6},\ 0.9999998,\ -5.9 \times 10^{-4})$ and the fourth is $(-1.2 \times 10^{-3},\ 0.99997,\ -7.9 \times 10^{-3})$: parallel to within about half a degree, but not parallel. ssik's dispatcher therefore found "no tier-0 (Pieper-class) match" and emitted its *tier-2 general 6R* solver, `ikgeo.general_6r`: the Raghavan–Roth elimination with the Manocha–Canny eigenvalue formulation. That solver makes no structural assumption about the chain and can return up to 16 real solutions for a general 6R arm; for a chain this close to the UR geometry one expects the usual eight postures among them, but the code never assumes that count. The artifact records `EXPECTED_MS_MEDIAN = 5.0` ms per solve.

Joint limits that the solver enforces. The artifact carries the URDF limits, in radians: shoulder pan $\pm 2\pi$, shoulder lift ± 4.712 ($\pm 270°$), elbow $\pm\pi$, and each wrist ± 4.189 ($\pm 240°$). Four of the six ranges are wider than one full turn, so the same physical posture can be represented by two (pan: up to three) different in-limit joint vectors that differ by 2π. The elbow range is exactly one turn, so the elbow never has an in-limit twin. This is the root of the 2π issue handled by the adapter.

Vocabulary. A *branch* is one geometrically distinct solution: two joint vectors that agree modulo 2π on every joint are the same branch. A 2π *twin* is a different representative of the same branch. ssik returns one representative per branch; the adapter's job is to pick the representative that sits nearest to the seed in *raw* joint values, because that is what the planner's continuity gate measures.

Algorithm 3: SsikArmBranches — `ssik_ik.ssik_arm_branches` and `rebranch_toward_seed`

---

function SsikArmBranches(arm, X_{world}, q_{seed}, `max_solutions`, `allow_rescue`)
  VerifyJointOrder(robot) ▷ once per robot: artifact joint order = URDF chain order
  $W \leftarrow$ world pose of `<arm>_ur_arm_base_link`, read live from PyBullet
  $T \leftarrow W^{-1}\, X_{\text{world}}$ ▷ ssik expects tool0 in the arm-base frame
  sols $\leftarrow$ ArtifactSolve(T; `max_solutions`, q_{seed}, `allow_rescue`) ▷ Alg. 4
  $B \leftarrow [\, q \text{ for } (q, \text{residual}) \in \text{sols}\,]$
  if $q_{\text{seed}} \neq$ None **then**
    $B \leftarrow [\, \text{Rebranch}(q, q_{\text{seed}}, \text{limits}) \text{ for } q \in B\,]$
    sort B by $\|q - q_{\text{seed}}\|_\infty$ ascending ▷ raw distance, the metric of Eq. 3
  end if
  return B ▷ empty = unreachable, or every branch outside the limits
end function

function Rebranch(q, seed, limits)
  for $i = 1 \ldots 6$ **do**
    $k \leftarrow \text{round}((\text{seed}_i - q_i)/2\pi)$ ▷ whole turns that bring q_i onto the seed's turn
    if $\text{lo}_i \leq q_i + 2\pi k \leq \text{hi}_i$ **then** $q_i \leftarrow q_i + 2\pi k$
    end if ▷ only when the twin is inside the limits
  end for
  return q
end function

The two red lines are the whole point of the adapter. ssik already sorts by distance to the seed, but with a *wrap-to-π* metric (its `seed_metric="wrap_linf"`), under which a joint at 3.0 rad and its twin at $3.0 - 2\pi = -3.28$ rad are at distance zero. The planner's continuity gate takes the raw difference, 6.28 rad, and would reject that step as a branch flip. Rebranch moves every joint onto

the seed's turn when the limits allow it, and the re-sort makes the first element the branch that is nearest in the sense the planner will actually test.

Algorithm 4: ArtifactSolve — the generated `solve()` in `asset/ssik/<arm>_ur_arm_ik.py`

---

function ArtifactSolve(T; `max_solutions`, q_{seed}, `allow_rescue`)
  candidates $\leftarrow$ SolveAlgebraic(T) ▷ Alg. 5; raw roots, no limits, may be empty
  verified $\leftarrow$ [ ]
  for q **in** candidates **do**
    if any entry of q is not finite **then**
      continue
    end if
    $r \leftarrow \|\mathrm{FK}(q) - T\|_F$ ▷ Frobenius norm of the 4×4 difference
    if $r \le 10^{-5}$ **then** append (q, r) to verified
    end if ▷ FK-closure gate; near-misses are dropped (no Newton polish)
  end for
  deduped $\leftarrow$ [ ]
  for (q, r) **in** verified **do**
    if some $(q', r') \in$ deduped has $\max_i |\mathrm{wrap}(q_i - q'_i)| \le 10^{-3}$ **then** keep the better one
    else append (q, r)
    end if
  end for ▷ one representative per branch, 2π twins collapsed
  if deduped $= \varnothing$ **and** `allow_rescue` **and** $\|p_T\| \le$ reach radius **then**
    deduped $\leftarrow$ RescueViaPerturbation(T) ▷ Alg. 6; reach radius = sum of link offsets
  end if
  sols $\leftarrow$ WrapToLimits(deduped) ▷ out-of-range joint: try $q_i + 2\pi k$, $k \in \{1, -1, 2, -2\}$
  sols $\leftarrow$ RespectLimits(sols) ▷ drop any solution with a joint still outside its range
  if $q_{\text{seed}} \ne$ None **then** sort sols by $\max_i |\mathrm{wrap}(q_i - q_{\text{seed},i})|$
  end if ▷ wrap-to-π metric
  if `max_solutions` given **then** truncate sols to the first `max_solutions`
  end if
  return sols ▷ each carries q, its FK residual, and a flag saying whether the rescue produced it
end function

Three consequences of this ordering are worth holding in mind. (i) The limit filter runs *after* dedup, so a branch whose only in-limit representative is a 2π twin still survives: WrapToLimits finds that twin. (ii) WrapToLimits prefers the smallest $|k|$, i.e. it biases toward the raw algebraic value, not toward the seed; the adapter's Rebranch corrects that afterwards. (iii) With `max_solutions` set (the goal solve in `_ik_dual_arm_at_frames` uses 8), the truncation happens under ssik's wrap metric, *before* the adapter re-sorts under the raw metric. Every other call site passes `None` and receives the full branch set.

Algorithm 5: SolveAlgebraic — Raghavan–Roth / Manocha–Canny core (`_solve_algebraic`)

---

function SolveAlgebraic(T)
  $T_{\text{dh}} \leftarrow T_{\text{pre}}^{-1}\, T\, T_{\text{post}}^{-1}$ ▷ URDF (POE) frames $\rightarrow$ calibrated DH chain
  $(P_{\sin}, P_{\cos}, P_1, Q) \leftarrow$ BuildPQ(T_{dh}) ▷ 14×9 / 14×8; DH constants inlined at build time
  $(E_{\sin}, E_{\cos}, E_1) \leftarrow N^\top (P_{\sin}, P_{\cos}, P_1)$ with $N^\top Q = 0$ ▷ removes the (q_1, q_2) terms; 6×9
  $(E_{\text{quad}}, E_{\text{lin}}, E_{\text{const}}) \leftarrow$ half-angle substitution $x_i = \tan(q_i/2)$ for $i = 3, 4, 5$
  $(M_{\text{quad}}, M_{\text{lin}}, M_{\text{const}}) \leftarrow$ stack the six equations with x_4-shifted copies ▷ 12×12 pencil in x_3
  (roots, vecs) $\leftarrow$ real eigenvalues of $M_{\text{quad}} x_3^2 + M_{\text{lin}} x_3 + M_{\text{const}}$ ▷ Möbius fallback if ill-conditioned

```
    out ← [ ]
    for (x_3, v) in zip(roots, vecs) do
        read q_4, q_5 from the eigenvector v                          ▷ back-substitution starts
        recover q_1, q_2 from the P/Q system, then q_6
        if back-substitution failed then
            continue
        end if
        append q_dh − θ_offset to out                                 ▷ back to the URDF joint zero
    end for
    return out                                                        ▷ at most 16 candidates; each is re-checked by FK in Alg. 4
end function
```

(Joint numbering in Alg. 5 is 1-based to match the text; the source uses 0-based names such as `eliminate_q0_q1`.) The one-time cost of choosing which joint to eliminate first was paid at `ssik build`; at run time the whole routine is dense linear algebra on small matrices.

Algorithm 6: RescueViaPerturbation — `ssik.refinement.rescue` (only when `allow_rescue=True`)

```
function RescueViaPerturbation(T)
    found ← [ ];   rng ← fixed seed                                   ▷ deterministic
    for i = 0 ... 15 do
        m ← (1, 2, 4, 10)[i mod 4]                                    ▷ escalating scale
        ΔT ← random small motion: translation ∼ 5 mm · m, rotation ∼ 5 mrad · m
        for q in ArtifactSolve(T ΔT; limits off, rescue off) do       ▷ off the ridge the algebra works
            q* ← LM-refine q toward T (analytic Jacobian, ≤ 20 iterations)
            if q* failed or ‖FK(q*) − T‖ > 10^−8 then
                continue
            end if
            if q* within 10^−4 rad (wrap-to-π) of something already in found then
                continue
            end if
            append q* to found, tagged refinement_used="lm"
        end for
    end for
    return found
end function
```

The rescue exists for a measure-zero situation: a reachable pose at which the Raghavan–Roth pencil loses rank, so the algebra extracts nothing even though solutions exist arbitrarily close by. It costs about sixteen ordinary solves plus refinement, roughly 40× a normal call according to the loader's own comment. That is why the planner passes `allow_rescue=False` inside every per-waypoint loop (tree extension, tracking, stitching, smoothing) and leaves it on only for the one-off endpoint solves: in a loop an unreachable waypoint should fail in about a millisecond, and a rescued, numerically refined solution may sit on a different branch than its neighbours anyway.

What an empty list means. `ssik_arm_branches` returns [ ] in exactly three cases: the tool pose is outside the arm's reach (no real roots close under FK), every branch has a joint outside its limits even after $\pm 2\pi$ wrapping, or – with rescue off – the pose sits on a rank-deficient ridge. The planner does not distinguish these; all three become `ik_failure` in the profile counters.

Where each call site sits.

caller	used for	seed	`max_solutions`	rescue
`_solve_dual_arm_pose_ik_ssik`	every waypoint	previous q	all	off
`_ik_dual_arm_at_frames`	goal from tool frames	placeholder start	8	on
`_ssik_pair_goal_branch_with_home`	branch pairing	none	all	on
`_solve_endpoint_dual_arm_ik_ssik`	goal repair (Alg. 10)	goal q	8	on

4.4 Growing one tree toward a pose

Algorithm 7 is used both for the "extend toward a random sample" step and for the "connect the other tree" step. It always walks the whole segment; it does not take a single ε step.

Algorithm 7: ExtendToward — `extend_toward`

function ExtendToward($\mathcal{T}$, v_{src}, T_{target})
  $v \leftarrow v_{\text{src}}$; $q \leftarrow \mathcal{T}.\text{conf}[v]$; reached $\leftarrow$ **true**; reason $\leftarrow$ "reached"
  define Stop(r): reason $\leftarrow r$; reached $\leftarrow$ **false**; leave the loop
  for T **in** Interp(v.pose, T_{target}; δ_p, δ_r) **do** [D-5]
    if IK enabled (stage ≥ 2) **then** [D-6]
      $q' \leftarrow$ SolveDualArmPoseIK(T, q) ▷ warm-seeded from the parent
      if $q' =$ None **then** Stop("ik_failure")
      end if
      if Jump(q', q) **then** Stop("continuity") [D-6]
      end if
      if stage $= 3$ **and** `joint_collision_fn`(q') **then** Stop("collision")
      end if
    end if
    if `collision_fn`(T) **then** Stop("collision") ▷ bar-only check
    end if
    $v_{\text{new}} \leftarrow$ new node(pose T, parent v); add to $\mathcal{T}$; cache conf q' and $\phi(T)$ [D-1]
    $v \leftarrow v_{\text{new}}$; $q \leftarrow q'$
  end for
  return (v, reached, reason) ▷ v is the last node that was accepted
end function

Because the loop appends every accepted waypoint before it fails, a blocked walk still leaves a chain of nodes reaching to the obstacle. That is the same behaviour as the baseline's Connect (which calls Extend repeatedly), but here it is also what the *first* extension does.

4.5 The bidirectional loop

Algorithm 8: PlanPoseBiRRT — `plan_pose_birrt`

function PlanPoseBiRRT(T_s, T_g, q_s, q_g)
  require q_s, q_g hold the bar at T_s, T_g and pass `joint_collision_fn` [D-9]
  (stage 1 checks only `collision_fn`(T_s), `collision_fn`(T_g))
  for attempt $= 1 \ldots$ `max_attempts` **do** [D-8]
    $\mathcal{T}_a \leftarrow$ tree rooted at (T_s, q_s, $\phi(T_s)$); $\mathcal{T}_b \leftarrow$ tree rooted at (T_g, q_g, $\phi(T_g)$) [D-1]
    for iter $= 1 \ldots$ `max_iterations` **do**
      if elapsed $\geq$ `max_time` **then**
        break

```
        end if
        (T_grow, T_other) ← (T_a, T_b) if |T_a| ≤ |T_b| else (T_b, T_a)                [D-4]
        if rand() < goal_sample_prob (= 0.05) then                                     [D-3]
            T_rand ← root pose of T_other                  ▷ cross-bias toward the other tree
        else
            T_rand ← SamplePose()          ▷ uniform box around the base, random Euler angles
        end if
        v_near ← arg min_{v∈T_grow} ‖φ(v.pose) − φ(T_rand)‖_2                          [D-2]
        (v_new, _, reason) ← ExtendToward(T_grow, v_near, T_rand)                      [D-5]
        record reason in extend_stop_reasons
        if v_new = v_near then
            continue
        end if                                                  ▷ no progress at all
        u_near ← arg min_{u∈T_other} ‖φ(u.pose) − φ(v_new.pose)‖_2
        (u_new, connected, _) ← ExtendToward(T_other, u_near, v_new.pose)              [D-5]
        if not connected then
            continue
        end if

    — the two trees now share the pose v_new.pose; order the path start → goal —
        if T_grow = T_a then
            S ← Retrace(v_new)                          ▷ root of T_a (start) → seam
            E ← Reverse(Retrace(u_new))                 ▷ seam → root of T_b (goal)
        else
            S ← Retrace(u_new);   E ← Reverse(Retrace(v_new))
        end if
        E ← E[1:]                                       ▷ drop the duplicated seam node
        poses ← poses of S followed by poses of E
        if IK enabled then
            confs ← Stitch(S, E, q_g)                                                  [D-7]
            if confs = None then
                continue
            end if                                  ▷ seam not joint-continuous; keep searching
        end if
        return (poses, confs)
    end for
                                          ▷ attempt exhausted: discard both trees and start fresh
end for
return (None, None)
end function
```

4.6 Stitching the seam

Each tree carries its own joint configurations, grown outward from its root by warm-seeded IK. The start tree's configurations therefore sit on the branch sheet of q_s and the goal tree's on the sheet of q_g. Even when the two trees meet at the same bar pose, the two configurations at the seam may differ by a branch flip. Algorithm 9 repairs this by re-solving one side from the other side's seam configuration.

Algorithm 9: Stitch — inner functions `_stitch_forward` / `_stitch_backward`

function STITCH(S, E, q_g)
  $Q_S \leftarrow$ cached confs of S; $Q_E \leftarrow$ cached confs of E
  $Q'_E \leftarrow$ REWALK(poses of E, seed $= Q_S[-1]$) ▷ forward: goal side on the start sheet
  if $Q'_E \neq$ NONE **then return** $Q_S \,\|\, Q'_E$
  end if
  $Q'_S \leftarrow$ REVERSE(REWALK(REVERSE(poses of S), seed $= Q_E[-1]$))
  backward: start side rebuilt on the goal sheet (the seed falls back to q_g when E is empty, i.e. the trees met at the goal root)
  if $Q'_S =$ NONE **or** JUMP($Q'_S[0]$, $Q_S[0]$) **then return** NONE
  end if ▷ must still end at the given start conf
  return $Q'_S \,\|\, Q_E$
end function

function REWALK(poses, seed)
  $q \leftarrow$ seed; out $\leftarrow [\,]$
  for T **in** poses **do**
    $q' \leftarrow$ SOLVEDUALARMPOSEIK(T, q)
    if $q' =$ NONE **or** JUMP(q', q) **or** `joint_collision_fn`(q') **then return** NONE
    end if
    append q' to out; $q \leftarrow q'$
  end for
  return out
end function

Note what a stitch failure does *not* do: it does not remove any node from either tree. The iteration simply continues, and the same seam can be hit again later from a different frontier. The failure is counted under `stitch_ik_failure`, `stitch_continuity_fail`, `stitch_collision` or `stitch_endpoint_mismatch` in the profile.

4.7 Sampling

SAMPLEPOSE (`sample_pose` with `goal_sample_prob`= 0 in the bidirectional caller) draws the bar position uniformly in a box centred on the robot base: ± 1.1 m in x and y (`workspace_xy`= 2.2), and $z \in [\max(0.05, z_{\text{base}}),\ \max(0.05, z_{\text{base}}) + 1.2]$. Orientation is built from three independent uniform angles in $[-\pi, \pi]$ (roll, pitch, yaw). This is *not* uniform on the rotation group, but no part of the planner depends on it being so. Note that there is no bias toward the goal pose in the random branch; the only bias is the 5 % cross-bias toward the *other tree's root* in Algorithm 8.

5 Core differences from the standard BiRRT

The tags below match the red [D-n] markers in the pseudocode.

D-1 Nodes are bar poses, not configurations.
A tree node stores a pose $T \in \mathrm{SE}(3)$ plus two side caches: the joint configuration that IK produced for it, and its feature vector $\phi(T)$. The search never samples joints. The joint path is a *by-product* that is grown alongside the pose tree, one warm-seeded IK per node.

D-2 Distance is measured on the bar's corners.
Nearest-neighbour search uses $\|\phi(\cdot) - \phi(\cdot)\|_2$ (Eq. 2), a Euclidean distance on eight projected corner points. Translation and rotation are combined through the geometry of the carried object instead of a hand-chosen weight.

D-3 No goal bias; a small cross-bias instead.
Random samples are drawn from a fixed box around the base. With probability 0.05 the

target is replaced by the *other* tree's root, which pulls the two trees toward each other. The single-tree planner (`plan_pose_rrt`, archival) used a 0.1 goal bias.

D-4 The smaller tree grows.
Instead of strict alternation, each iteration extends whichever tree currently has fewer nodes, then asks the other tree to connect. The path is always assembled start → goal regardless of which tree grew.

D-5 Extend is a full greedy walk with partial progress kept.
Both the extension toward the sample and the connection toward the new frontier walk the entire interpolated segment at resolution (δ_p, δ_r), stopping at the first rejected waypoint and keeping every accepted one. The baseline takes one fixed step for the first move. "Connected" therefore means the other tree walked all the way to the exact pose of the new frontier (the last interpolated pose is the target itself), not that it came within a tolerance.

D-6 Four validity gates per waypoint, one of them kinematic.
A waypoint is accepted only if (a) both arms have an analytical IK solution within joint limits, (b) the nearest solution is within $\tau = 10°$ per joint of the previous waypoint's configuration, (c) that configuration is collision-free for the whole cell (stage 3), and (d) the bar alone is collision-free at that pose. Gate (b) has no counterpart in the baseline; it is what turns a pose path into a joint path that can be executed without a branch flip, and it is the dominant reason extensions stop (Section 8).

D-7 Meeting at a pose is not enough; the seam is re-solved.
The two trees carry joint configurations from two different roots, so when they meet the joint values may disagree even though the poses are identical. The goal-side configurations are rebuilt from the start-side seam configuration (or, failing that, the start side from the goal branch). If neither rewalk passes the continuity and collision gates, the candidate path is thrown away and the search continues with both trees intact.

D-8 Restarts with fresh trees.
Up to `max_attempts` independent attempts, each with its own time budget (`max_time`) and iteration cap. Trees are not carried over between attempts; the random generator is.

D-9 Both endpoints must be given as configurations.
The goal tree needs a root configuration, so q_g has to be solved before the search. Because the whole path must stay on one branch sheet, the endpoint IK cannot be chosen independently for start and goal; the pre-processing in Section 6 exists to make q_s and q_g mutually reachable.

6 Around the search: endpoints, stages, smoothing

6.1 Stages

`plan_constrained_dual_arm(stage=...)` switches gates on incrementally, which is how the planner was built and how it is still debugged:

stage	IK per waypoint	continuity gate	collision
1	off	off	bar only (`collision_fn`)
2	on	on	bar only
3	on	on	bar + full cell (`joint_collision_fn`)

Stage 3 is the production setting. A stage-1 success returns `info["pose_only_success"]` and no joint path.

6.2 Getting the two endpoints (`derive_start=True`)

For the M1 movement the Rhino export carries the goal as two tool0 frames and leaves the start joints as a placeholder, so the entry point computes both endpoints itself. The order is *goal first*: the goal fixes the grasp transforms, then a start is found that the goal can reach.

1. **Goal IK.** Solve each arm at its goal tool frame, nearest analytical branch to the (placeholder) seed; FK at that configuration gives T_g and, with the authored attachment frame, G_L and G_R.
2. **Goal/home branch pairing** (`_ssik_pair_goal_branch_with_home`). Enumerate all legal IK branches at the goal and at every candidate home pose (three carry anchors $\times$ their roll/yaw variants), rank each goal branch by its $\|\cdot\|_\infty$ distance to the nearest home branch after 2π re-branching, and take the first of the top-4 $\times$ 4 pairs that is FK-valid and collision-free. Measured on B6, independently picked branches were 104–230° apart; the best pairing is 38°/67°.
3. **Goal collision repair.** If the paired goal collides, `solve_endpoint_dual_arm_ik` is asked for a different configuration at the *same* bar pose and grasps. Its signature still carries `rng` and `max_attempts` from the gradient era, where PyBullet's numeric IK converges to whichever branch basin its seed lands in, so re-seeding at random was the only way to reach another branch. On the ssik backend both arguments are ignored: the branch set is finite and enumerable, so the ssik variant scans it deterministically (Alg. 10). "Re-solve" therefore means "walk the list of left×right branch pairs, nearest to the current goal first, and take the first one that holds the bar and is collision-free". If none does, the plan fails with `goal_in_collision`.
4. **Backward-tracked start** (Alg. 11): walk the bar from T_g toward candidate home poses with the same warm-seeded IK and continuity gate the tree uses. Whatever it arrives at is on the goal's own branch sheet by construction.
5. **Direct-path shortcut.** If one of those walks is also collision-free at every waypoint, its reversal *is* the M1 path and the search is skipped entirely; the returned `info["planner"]` then reads `"direct_path"`. The smoother is not run in that case.
6. **Cold fallback.** `derive_constrained_start`, the older endpoint sweep, is used when tracking finds nothing: it picks a holdable home pose by endpoint IK alone.

Algorithm 10: EndpointIK — `_solve_endpoint_dual_arm_ik_ssik` (goal collision repair)

---

function EndpointIK(T, G_L, G_R, q_{seed}, `collision_fn`)
  $B_L \leftarrow$ SsikArmBranches(L, TG_L, $q_{\text{seed},L}$, `max_solutions`= 8, rescue on)
  $B_R \leftarrow$ SsikArmBranches(R, TG_R, $q_{\text{seed},R}$, `max_solutions`= 8, rescue on)
  if $B_L = \varnothing$ **or** $B_R = \varnothing$ **then return** None
  end if ▷ an arm cannot reach its grasp at all
  pairs $\leftarrow [\,(q_L, q_R)$ for $q_L \in B_L,\ q_R \in B_R\,]$ ▷ up to $8 \times 8 = 64$ combinations
  sort pairs by $\|(q_L, q_R) - q_{\text{seed}}\|_2$ ascending ▷ stay close to the goal the pairing chose
  for (q_L, q_R) **in** pairs **do**
    $q \leftarrow (q_L, q_R)$
    if not `validate_dual_arm_bar_pose`(q, T) **then**
      continue
    end if ▷ PyBullet FK gate
    if `collision_fn`(q) **then**
      continue
    end if ▷ full-cell check
    return q ▷ first pair that is reachable, consistent, and free
  end for
  return None ▷ every branch pair is either FK-inconsistent or colliding
end function

Two details. The pair order uses the Euclidean norm on the raw 12-vector, not the $\|\cdot\|_\infty$ of the continuity gate, and nothing here checks continuity: the repaired goal may sit on a different branch than the one `_ssik_pair_goal_branch_with_home` chose, in which case the backward-tracked start derivation (next step) starts from the repaired goal and finds a home on *its* sheet. With the gradient backend the same function instead loops `max_attempts` times over `solve_dual_arm_pose_ik`, seeding attempt 0 from q_{seed} and every later attempt from a uniform random 12-vector in $[-\pi, \pi]$.

Algorithm 11: DeriveStartTracked — `derive_constrained_start_tracked` (condensed)

function DeriveStartTracked(T_g, q_g)
  variants ← every selected anchor's canonical orientation, then roll/yaw variants smallest-first
  deltas ← grid in the sweep box (±0.3 m, step 0.1 m), sorted nearest-first
  budget ← 120 s split evenly across the selected anchors; first_start, best_partial ← None
  for (label, T_{anchor}) **in** variants **do**
    for Δ **in** (all deltas **if** canonical **else** first 60) **do**
      if this anchor's time share is spent **then**
        break
      end if
      $T_h \leftarrow T_{\text{anchor}}$ shifted by Δ
      (track, ok) ← TrackDirectPath(T_g, q_g, T_h; screening step 0.01 m / 0.025 rad)
        = Rewalk without the collision gate; returns the accepted prefix and whether it reached T_h
      if not ok **then**
        best_partial ← the broken track whose last pose is farthest from T_g
        continue
      end if
      if `joint_collision_fn`(track[-1].conf) **then**
        continue
      end if ▷ the arrival is a plan endpoint
      if no interior waypoint of track collides (scanned every 3rd first) **then**
        re-track at the requested (δ_p, δ_r) if finer than screening; on success
        return (T_h, arrival conf, `direct_path` = track) ▷ caller skips the search
      end if
      if first_start = None **then** first_start ← (T_h, arrival conf)
      end if ▷ blocked mid-way; keep as start-only
    end for
  end for
  if first_start ≠ None **then return** first_start
  end if
  if best_partial reaches ≥ 2 cm from T_g **then** ▷ M0 will cover the remaining gap
    return its last collision-free waypoint still ≥ 2 cm from T_g, flagged `partial`
  end if
  return (None, None)
end function

6.3 Smoothing

The raw path is a chain of interpolated waypoints with sharp corners at every tree node. The smoother (Algorithm 12) is a classic random shortcut in *pose* space with the joint path re-derived for each accepted shortcut, so the same three gates (IK, continuity, collision) hold on the result.

Algorithm 12: SmoothPosePath — `smooth_dual_arm_pose_path`

function SmoothPosePath(poses, confs)
  cost $\leftarrow \sum_i d(T_i, T_{i+1})$ (Eq. 2); corners $\leftarrow$ indices where the feature-vector direction changes
  for iter $= 1 \ldots$ `smooth_max_iterations` $(= 100)$, within `smooth_max_time` $(= 10$ s$)$ **do**
    if fewer than 3 corners **then**
      break
    end if
    pick two corners $i < j$ at least two corners apart
    shortcut $\leftarrow$ Interp$(T_i, T_j; \delta_p, \delta_r)$; candidate $\leftarrow$ poses$[:i] \,\|\, $ shortcut $\,\|\,$ poses$[j{+}1:]$
    if cost(candidate) $\geq$ cost **then**
      continue
    end if
    $Q \leftarrow$ Rewalk(candidate$[i:]$, seed = confs$[i]$) ▷ IK + continuity + collision from i to the end
    if $Q =$ None **then**
      continue
    end if
    poses $\leftarrow$ candidate; confs $\leftarrow$ confs$[:i] \,\|\, Q$; recompute cost and corners
  end for
  return (poses, confs)
end function

The rewalk starts at i and runs to the *end* of the path, not only over the shortcut, because the configurations after j were seeded from the old waypoints and may no longer be continuous with the new ones.

7 Parameters and their defaults

The same knob often has a different default at each layer. The values below are what the code carries today.

parameter	core default	api default	headless M1	meaning
`position_res` δ_p	0.05 m	0.01 m	0.002 m	extend step; also the goal tolerance
`rotation_res` δ_r	0.10 rad	0.025 rad	0.002 rad	extend step; also the goal tolerance
`max_time`	30 s	30 s	CLI `-max-time`	per attempt
`max_iterations`	2000	2000	—	per attempt
`max_attempts`	5	5	—	fresh tree pairs
`goal_sample_prob`	0.05	(not exposed)	—	cross-bias to the other root
`workspace_xy` / `_z`	2.2 m / 1.2 m	(not exposed)	—	sampling box
τ (continuity)	10°	10°	—	the constant in Eq. 3
`dist_metric`	`"feature"`	(not exposed)	—	Eq. 2
smoothing	—	100 iters / 10 s	—	smoother iteration and time caps
tracked-start budget	120 s	(not exposed)	—	split evenly across anchors
screening step	0.01 m / 0.025 rad	(not exposed)	—	never finer than δ_p, δ_r
`start_home_anchor`	—	all	`-cdfm-home-anchor`	horizontal / vertical / back

Here *core* is `plan_pose_birrt` (and, for the last three rows, `derive_constrained_start_tracked`), *api* is `plan_constrained_dual_arm`, and *headless M1* is what `scripts/headless_bar_action_planner.py` passes for an M1 movement.

δ_p and δ_r reach four places: the tree extension, the connect walk, the smoother's shortcuts, and the delivered direct path. Lowering them makes every one of those denser; it also makes each extension cost more IK solves and collision checks.

8 What the benchmark measured

These numbers are copied from `benchmark/analysis_night1.md` (run `night1`: 24,000 runs over 16 bars, 800 goals, 3 starts per goal, 10 seeds; each run is a coarse pass at 0.1 m / 0.1 rad, 30 s and a fine pass at 0.002 m / 0.002 rad, 300 s). They are observations, not conclusions, and that document states nothing in it has been acted on.

- Solved: 16.3 % of runs; 28.0 % of problems within ten seeds. Median solved time 143 s (fine), 21 s (coarse).
- Why extensions stop: **continuity 75.4 %**, IK failure 18.5 %, collision 3.7 %. The gate in D-6(b) is the main brake, not the environment.
- Where the time goes: collision checking 36.8 % (about 12.9 ms per call), IK 33.0 %, stitching 13.0 %, smoothing 9.8 %.
- The tracked start derivation found a collision-free direct path (and skipped the search) for 34.9 % of goals and used its whole 120 s budget at the median; the survey found that 73 % of goals have at least one straight-line-solvable start.
- On problems where a straight-line motion provably exists, the search still failed 56 % of the time, for the same continuity reason.

Where the "why extensions stop" numbers come from. The three percentages are lifted from Finding 1 of `benchmark/analysis_night1.md`; the raw run data behind them (`runs/night1/summary.json` under the benchmark data root) is not in the repository and was not available on the machine this report was written on, so the shares could not be recomputed here. What *can* be traced is the counter itself:

1. `plan_pose_birrt` keeps one `Counter` named `extend_stop_reasons`. It is incremented **once per call to ExtendToward**, with the walk's stop reason: `reached`, `collision`, `ik_failure` or `continuity` for the growth walk (Alg. 8 line 17), the same names prefixed `connect_` for the connect walk, and the four keys `stitch_{ik_failure, continuity_fail, collision, endpoint_mismatch}` from Alg. 9.
2. `update_debug_tree` copies the counter into the caller's `debug_tree_out` dict, and the entry point returns that dict as `info["profile"]`. The histogram is the entry `"extend_stop_reasons"` of that dict.
3. The benchmark (`benchmark/cdfm.py`) stores that dict on every pass record, and `benchmark/summarize.py` adds the dicts of *both* passes (coarse and fine) of *every* run (solved or not) into one histogram per group.

Two consequences for reading the table. A "continuity" entry is one *walk* that ended on the gate, not one waypoint: a walk that added twenty nodes and then hit a branch flip counts exactly like one that died on its first step. And the analysis document does not say which keys form its denominator; the dashboard plots every key as-is, so `reached`, `connect_*` and `stitch_*` are presumably what "everything else, 2.4 %" contains. If so, connect walks are rare, which would mean most growth walks make no progress at all before the gate fires. That reading should be confirmed against the summary file before anything is built on it.

Reading the numbers against the pseudocode. Every one of the three gates in Algorithm 7 is evaluated *per waypoint*, so at $\delta = 0.002$ a single extension across the workspace is hundreds of IK solves and collision checks, and a single rejected waypoint ends it. The continuity share suggests that "nearest branch to the seed" alone does not keep the walk on one sheet often enough; whether that is a threshold question, a sampling question, or an IK question is exactly what the stage switches and the profile counters are there to separate.

9 Reading guide

Suggested order for reading the code with this report open. Line numbers are from the tree at commit `cd239d5` (29 September 2026).

read	at	this report
`joint_step_exceeds_threshold`	`core.py:186`	Eq. 3
`get_bar_feature_points`, `pose_to_feature_vec`	`core.py:212`, `:412`	Eq. 2
`sample_pose`, `nearest_node`	`core.py:541`, `:583`	D-2, D-3
`build_cfab_pose_collision_fn`	`core.py:645`	bar-only predicate
`validate_dual_arm_bar_pose`	`core.py:905`	FK gate in Alg. 2
`_solve_dual_arm_pose_ik_ssik`	`core.py:1055`	Alg. 2
`ssik_arm_branches`, `rebranch_toward_seed`	`ssik_ik.py:145`, `:118`	Alg. 3
`ssik_inprocess.solve`	`keyframe/ssik_inprocess.py:99`	loader
artifact `solve`, `_solve_algebraic`	`asset/ssik/left_ur_arm_ik.py :1066`, `:817`	Algs. 4, 5
`wrap_to_limits`, `nearest_to_seed`	`ssik/postprocess.py:99`, `:157`	limit and seed filters
`rescue_via_T_perturbation`	`ssik/refinement/rescue.py:57`	Alg. 6
`extend_toward`	`core.py:2285`	Alg. 7
`plan_pose_birrt`	`core.py:2775`	Alg. 8, 9
`_solve_endpoint_dual_arm_ik_ssik`	`core.py:1292`	Alg. 10
`track_direct_path`, `first_blocked_index`	`core.py:1824`, `:1905`	Alg. 11
`derive_constrained_start_tracked`	`core.py:1936`	Alg. 11
`reconstruct_joint_path_for_pose_path`	`core.py:2448`	REWALK
`update_debug_tree`	`core.py:2511`	stop-reason export (Section 8)
`smooth_dual_arm_pose_path`	`smooth.py:20`	Alg. 12
`_ssik_pair_goal_branch_with_home`	`api.py:254`	Section 6 step 2
`_derive_constrained_start_for_plan`	`api.py:383`	Section 6
`plan_constrained_dual_arm`	`api.py:753`	entry point

Two things the reader should not look for, because they are absent by design: there is no goal-region test in the bidirectional planner (the trees meet at an exact pose, D-5), and there is no node pruning or rewiring of any kind (a failed stitch leaves both trees untouched, D-7).